

MILESTONE 3: Backend – FastAPI Integration ([main.py](#))

This milestone focuses on implementing the FastAPI-powered backend for **PocketSmart AI**. It acts as the bridge between the frontend interface and the Gemini-based AI recommendation engine defined in [gemini_utils.py](#). The backend is designed with modular architecture, providing clean and maintainable API endpoints for planners across Home, Party, and Jewelry domains. It also includes features like CORS setup for cross-origin requests, session handling for personalized user experience, and structured input/output models to ensure smooth AI interaction and reliable frontend rendering.

- **Activity 3.1: Define FastAPI Routes**

- Home Planner ([/generate-home](#)): Accepts home-related preferences and budget to return tailored product suggestions.

```
@app.post("/home-budget")

async def plan_home_budget(

    budget_input: HomeBudgetInput,

    request: Request,

    current_user: UserInDB = Depends(get_current_active_user)

):

    """Generate home budget recommendations"""

    # Store last budget planning in session data

    if current_user.username in active_sessions:

        active_sessions[current_user.username].user_data["last_home_budget"] = {

            "timestamp": datetime.utcnow().isoformat(),

            "budget": budget_input.total_budget,

            "requirements": {

                "lights": budget_input.num_lights,

                "fans": budget_input.num_fans,

                "furniture": budget_input.num_furniture,

                "dining_tables": budget_input.num_dining_tables

            }

        }

    }
```

```

# Get recommendations

result = get_home_recommendations(budget_input)


# Save to history

save_to_history(

    username=current_user.username,

    recommendation_type="home",

    input_data=budget_input.dict(),

    result=result

)


return result

```

- Party Planner ([/generate-party](#)): Processes party details and guest count to recommend venue, food, and decoration items.

```

# Update the party-budget endpoint
@app.post("/party-budget")
async def plan_party_budget(
    budget_input: PartyBudgetInput,
    request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate party budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_party_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "party_type": budget_input.party_type,
            "guests": budget_input.num_guests
        }

    # Get recommendations
    result = get_party_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,

```

```
        recommendation_type="party",
        input_data=budget_input.dict(),
        result=result
    )
```

```
    return result
```

- Jewelry Planner (</generate-jewelry>): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

```
@router.post("/jewelry-budget")
```

```
async def plan_jewelry_budget(
```

```
    budget_input: JewelryBudgetInput,
```

```
    current_user: User = Depends(get_current_active_user)
```

```
):
```

```
    total_budget = float(budget_input.total_budget)
```

```
    occasion = budget_input.occasion
```

```
    preferences = budget_input.preferences
```

```
    encoded_preferences = json.dumps(preferences)
```

```
    result_message = None
```

```
    # Generate jewelry recommendations with optional outfit image
```

```
    budget_input_data = JewelryBudgetInput(
```

```
        total_budget=total_budget,
```

```
        occasion=occasion,
```

```
        encoded_preferences=encoded_preferences
```

```
    )
```

```
    image_path = None
```

```
    if budget_input.image:
```

```
        image_path = save_upload_file(budget_input.image)
```

```

# Store last budget planning in session data

if current_user.username in active_sessions:

    active_sessions[current_user.username]["last_jewelry_budget"] = {

        "timestamp": datetime.utcnow().isoformat(),

        "budget_input": budget_input_data,

        "image_path": image_path

    }

# Get recommendations

result = get_jewelry_recommendations(budget_input, image_path)

# Save to history with image info

if current_user.username:

    result_data = budget_input_data

    if image_path:

        result_data["image"] = image_path

    save_to_history(

        username=current_user.username,

        category="jewelry-history",

        input_data=result_data

    )

return result

@router.post("/jewelry-budget")

```

```

async def plan_jewelry_budget(
    budget_input: JewelryBudgetInput,
    current_user: User = Depends(get_current_active_user)
):
    total_budget = float(budget_input.total_budget)

    occasion = budget_input.occasion

    preferences = budget_input.preferences

    encoded_preferences = json.dumps(preferences)

    result_message = None

    # Generate jewelry recommendations with optional outfit image

    budget_input_data = JewelryBudgetInput(
        total_budget=total_budget,
        occasion=occasion,
        encoded_preferences=encoded_preferences
    )

    image_path = None

    if budget_input.image:
        image_path = save_upload_file(budget_input.image)

    # Store last budget planning in session data

    if current_user.username in active_sessions:
        active_sessions[current_user.username]["last_jewelry_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),

```

```
        "budget_input": budget_input_data,  
        "image_path": image_path  
    }  
}
```

```
# Get recommendations
```

```
result = get_jewelry_recommendations(budget_input, image_path)
```

```
# Save to history with image info
```

```
if current_user.username:
```

```
    result_data = budget_input_data
```

```
    if image_path:
```

```
        result_data["image"] = image_path
```

```
    save_to_history(  
        username=current_user.username,  
        category="jewelry-history",  
        input_data=result_data  
    )
```

```
    return result
```

```
@router.get("/recommendation-history")
```

```
@router.get("/recommendation-history")
```



```
return {"history": []}
```

```
# Sort history by timestamp (newest first)
```

```
history = sorted(
```

```
    user_recommendations[current_user.username],
```

```
    key=lambda x: x.timestamp,
```

```
    reverse=True
```

```
)
```

```
# Convert to dict for JSON response
```

```
history_data = []
```

```
for item in history:
```

```
    history_data.append({
```

```
        "id": item.id,
```

```
        "timestamp": item.timestamp,
```

```
        "type": item.recommendation_type,
```

```
        "input": item.input_summary,
```

```
        "summary": item.result_summary
```

```
    })
```

```
return {"history": history_data}
```

- User Auth ([/register](#), [/login](#), [/logout](#), [/token](#)): Manages user registration, authentication, and session control.
- Session Info ([/session-info](#), [/session-data](#)): Retrieves user-specific session data for personalized AI interaction.

- **Activity 3.2: Modular Architecture Setup**

[/recommendations-details](#): Returns detailed AI-generated product recommendations based on user budget, preferences, and selected category (Home, Party, or Jewelry).

```

# Step 6: Add an endpoint to get details for a specific recommendation

@app.get("/recommendation-details/{recommendation_id}")

async def get_recommendation_details(

    recommendation_id: str,

    request: Request,

    current_user: UserInDB = Depends(get_current_active_user)

):

    """Get the full details of a specific recommendation"""

    if current_user.username not in user_recommendations:

        raise HTTPException(status_code=404, detail="No recommendations found")


    # Find the recommendation with the given ID

    for item in user_recommendations[current_user.username]:

        if item.id == recommendation_id:

            return {

                "id": item.id,

                "timestamp": item.timestamp,

                "type": item.recommendation_type,

                "input": item.input_summary,

                "full_result": item.full_result

            }

        raise HTTPException(status_code=404, detail="Recommendation not found")

```

- **Activity 3.3: CORS & Static Routing**

/history: Retrieves the user's past recommendation queries and results for review or re-use.

```

# Step 7: Add a new endpoint to serve the history page

```

```

@app.get("/history", response_class=HTMLResponse)

async def history_page(request: Request, current_user: UserInDB =
Depends(get_current_active_user)):

    """History page to view past recommendations"""

    return templates.TemplateResponse("history.html", {"request": request, "user":
current_user})

```

- **Activity 3.4: Startup and main function**

/startup: Initializes essential application services and loads configuration settings when the FastAPI server starts.

```
@app.on_event("startup")
```

```
async def setup_session_cleanup():
```

```
    """Background task to clean up expired sessions"""
```

```
async def cleanup_expired_sessions():
```

```
    while True:
```

```
        current_time = datetime.utcnow()
```

```
        # Check for sessions that have been inactive for more than 30 minutes
```

```
        expired_sessions = {
```

```
            username for username, session in active_sessions.items()
```

```
            if (current_time - session.last_activity).total_seconds() > 1800 # 30 minutes
```

```
        }
```

```
        # Remove expired sessions
```

```
        for username in expired_sessions:
```

```
            if username in active_sessions:
```

```
                print(f'Removing expired session for {username}')
```

```
                del active_sessions[username]
```

```
        # Wait for 5 minutes before checking again
```

```
await asyncio.sleep(300)
```

```
# Start the background task
```

```
asyncio.create_task(cleanup_expired_sessions())
```

`__main__`: Entry point of the FastAPI application that starts the server using `uvicorn` when the script is run directly.

```
# Main entry point
```

```
if __name__ == "__main__":
```

```
    import uvicorn
```

```
    print("Starting PocketSmart: AI Budget Planner...")
```

```
    uvicorn.run(app, host="0.0.0.0", port=8000)
```